ByteBoost Workshop: Project Description

Shape-Constrained Boosted Symbolic Regression for Interpretable Neural Scaling Laws: A Cross-Testbed Investigation

---

1 Field of science

Computer Science: Machine Learning and Artificial Intelligence, specifically the foundations of large language models (neural scaling laws) and interpretable machine learning via symbolic regression. Secondary field: High-Performance Computing and computer architecture (cross-platform performance characterization).

2 Abstract / summary

Training a modern AI language model can cost millions of dollars, so researchers use scaling laws to predict performance from model size and training data before committing resources. These equations guide major hardware and budget decisions, but their mathematical form is usually chosen manually and then fitted to measured results.

Our project uses symbolic regression to discover the scaling-law equation directly from data. Symbolic regression searches many candidate formulas and returns an interpretable equation rather than a black-box model. Unconstrained searches, however, often produce formulas that fit existing measurements but extrapolate unrealistically, predicting negative error or worse performance with more data.

We therefore require every candidate formula to satisfy admissibility axioms: larger models and additional data cannot increase error; improvements must diminish with scale; error must remain positive; and performance must approach an irreducible floor. We mathematically certify these properties across the full domain of interest, not only at measured points (Sections 4.2, 4.4 and 4.5).

The formula is built incrementally from a standard scaling law, adding one correction at a time while preserving all guarantees (Sections 4.3 and 4.4). We also prove that the corrections improve fit without changing the irreducible error floor or the asymptotic rate of improvement. The primary search path *penalizes* axiom violations inside a genetic-programming fitness and drives them toward zero while bounding any remaining shortfall (Section 4.6); a second path *rejects* inadmissible corrections outright and is a workshop extension of the soft engine (Sections 4.3 and 4.4).

The project has two computational bottlenecks. Model training will use Cerebras CS-3 systems on PSC's Neocortex platform and will be compared with completed NVIDIA GPU runs (Sections 3.1 and 3.2). Formula discovery is a large CPU-bound search dominated by interval-arithmetic certification. Because it is highly parallel and relies mainly on scalar or imperfectly vectorized Python (with optional JAX acceleration), Stony Brook's high-core-count AMA27 AmpereOne cluster is well suited to the workload (Sections 3.1 and 4.4).

Deliverables will include a certified scaling-law formula fitted to our training data, an open-source discovery library, and a concise methods report (Section 5.2). An initial set of trained models is already public (Section 3.5).

3 Key requirements

3.1 Testbed hardware (requested)

Code skeleton: `src/modeling/training/`, `src/systems/hpc/`, `src/search/`

- **Neocortex (PSC):** Cerebras CS-3 wafer-scale systems for the model-pretraining grid in Sections 3.4 and 3.5 (`https://www.psc.edu/resources/neocortex/`). Each CS-3 is built around a Wafer Scale Engine 3 (WSE-3) with on-chip SRAM and extreme memory bandwidth, so a model in our N range can stay on a single wafer without multi-GPU model parallelism. We use Neocortex only for pretraining; loss curves are compared to our existing NVIDIA GPU runs (Section 3.2).
- **AMA27 (Stony Brook Supercomputing Center / ACCESS):** Arm-based AmpereOne A192-32M cluster for the CPU-bound symbolic-regression search of Section 4.3 and the certification inner loop of Section 4.4. Per the ACCESS resource description, phase 1 deploys 312 single-socket compute nodes, each with a 192-core AmpereOne A192-32M CPU and 384 GB DDR5, linked by 200 Gbps NDR InfiniBand, with a 5 PB GPFS filesystem (`https://support.access-ci.org/affinity-groups/ama27`). AMA27 is the Stony Brook Arm testbed featured in ByteBoost 2.0 (in place of the retired Ookami A64FX system). We do *not* rely on A64FX-style 512-bit SVE or HBM: the GP/IA workload is population-parallel and largely scalar Python DualInterval evaluation, so the useful AMA27 properties are Arm ISA diversity versus our AMD EPYC (x86) baselines, high core counts per node, and dense node count for concurrent candidate evaluation.

3.2 Baseline hardware (already available to us)

Code skeleton: `src/systems/hpc/profiling.py`, `src/modeling/training/trainer.py`

GPU baselines (pretraining comparisons): 200 NVIDIA GH200 superchips on DeltaAI at NCSA, plus a local four-GH200 cluster. **CPU baselines (search/certification comparisons):** three 256-core AMD EPYC (x86) machines, against which we will profile the AMA27 AmpereOne port.

3.3 Software and tools

Code skeleton: `src/modeling/training/`, `src/search/`, `src/constraints/certificates/`

PyTorch 2.x for pretraining (distributed data-parallel training; optional mixed precision), with `uv` for environment management and Weights & Biases for experiment logging [1]. Symbolic search uses `gplearn` [2] with a custom fitness that adds soft interval-arithmetic axiom penalties (Section 4.6); an optional PySR backend provides an unconstrained alternate engine for ablation. Certificates are evaluated with a pure-Python forward-mode *DualInterval* interval-arithmetic layer (Section 4.4); JAX with custom JVPs is an optional acceleration path, not required. A hard reject filter that discards inadmissible candidates during search (Section 4.4) is a workshop extension built on the same certificate primitives.

3.4 Models

Code skeleton: `src/modeling/models/config.py`, `src/modeling/models/transformer.py`

Decoder-only transformer language models (a compact post-LN causal stack suitable for scaling grids), spanning roughly 10M–1.4B parameters, across several tokens-per-parameter ratios ($M = D/N$). Workshop ports may adopt LLaMA-style blocks [3], FlashAttention, and μP / μTransfer [4]; the core method only requires consistent (N, D, L) measurements in the Step Law [5] / overtraining [6] regimes.

3.5 Datasets

Code skeleton: `src/scaling/data/scaling_dataset.py`, `src/scaling/data/corpora.py`

Pretraining corpora used to generate new grid points include Wikipedia [7], RedPajama [8], and related open text mixtures (optional FineWeb-Edu [9] for workshop extensions). An already-public `colinear_scaling_models` collection on Hugging Face [10] supplies trained checkpoints and loss curves from an initial DeltaAI grid for the symbolic-regression track.

4 Method summary (technical)

Code skeleton: `src/scaling/`, `src/constraints/`, `src/search/`; map: `src/README.md`

4.1 Setup

Code skeleton: `src/scaling/setup/configuration.py`, `src/scaling/setup/constants.py`

Let N be the parameter count, D the training-token count, and $\mathcal{H} = \{h_1, \dots, h_m\}$ a finite set of remaining hyperparameters (in the concrete 4D grids used here, typically width-multiplier / weight-decay style axes and learning rate). For each coordinate $h \in \{N, D\} \cup \mathcal{H}$, let H_h be its finite set of admissible values and $\phi_h \colon H_h \to \mathbb{R}_{>0}$ a strictly monotone preprocessing map; write $\phi_i \coloneqq \phi_{h_i}$ for brevity. In the search implementation, genetic-programming features are the logarithms $\log \phi_h(h)$ (so stage corrections are fit in log-coordinates and mapped back to raw-scale derivatives by the chain rule; Section 4.4). The configuration space is the product $\mathbb{H} \coloneqq \prod_{h \in \{N,D\} \cup \mathcal{H}} H_h$, with typical element $\mathbf{h} \in \mathbb{H}$ (so N, D, h_i denote the corresponding coordinates of $\mathbf{h}$). Let $L \colon \mathbb{H} \to \mathbb{R}$ be validation loss; we seek $\hat{L} \approx L$. We are given a labeled dataset $\mathcal{D} = \{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^{n} \subset \mathbb{H} \times \mathbb{R}$. For each scale variable $x \in \{N, D\}$ set $x_{\min} \coloneqq \min H_x > 0$ and $x_{\max} \coloneqq \max H_x$, treat x as continuous on $[x_{\min}, \infty)$, and take all derivatives with the other coordinates held fixed.

4.2 Admissibility axioms

Code skeleton: `src/constraints/axioms/admissibility.py`

$\hat{L}$ is *admissible* ($\hat{L} \in \mathcal{S}$) if it satisfies (A1)–(A6) below. Where an axiom mentions a scale variable x, it is imposed for each $x \in \{N, D\}$ with the other coordinates held fixed.

(A1) **Monotone decrease.** $\dfrac{\partial \hat{L}}{\partial x} \le 0$ for all $x \ge x_{\min}$.

(A2) **Diminishing returns.** $\dfrac{\partial^2 \hat{L}}{\partial x^2} \ge 0$ for all $x \ge x_{\min}$.

(A3) **Irreducible loss.** There is a positive *joint floor* L_∞ and, along each axis, a *marginal floor* L_x^∞:

(i) *Joint floor.* There exists $L_\infty > 0$ such that $\hat{L}(\mathbf{h}) > L_\infty$ for every $\mathbf{h} \in \mathbb{H}$.

(ii) *Marginal floor.* For each $x \in \{N, D\}$ and every fixed choice of the other coordinates, the single-axis limit

$$L_x^\infty \coloneqq \lim_{t \to \infty} \hat{L}(\mathbf{h})\big|_{x=t} \tag{1}$$

exists and is finite, and satisfies $L_x^\infty \ge L_\infty$.

Equality $L_x^\infty = L_\infty$ holds only in the joint limit (all scale variables $\to \infty$ together); along a single axis the marginal floor is in general strictly larger.

(A4) **Asymptotic power-law decay.** Relative to the marginal floor (1),

$$\lim_{t\to\infty} \frac{\log\big(\hat{L}(\mathbf{h})\big|_{x=t} - L_x^\infty\big)}{\log t} \;=\; c_x \qquad \text{for some } c_x < 0. \tag{2}$$

(A5) **Positivity.** $\hat{L}(\mathbf{h}) > 0$ for all $\mathbf{h} \in \mathbb{H}$.

(A6) **Graceful saturation.** $\hat{L}$ is continuous and differentiable (C^1) in x on $[x_{\min}, \infty)$, with $\hat{L}(x_{\min}, \cdot)$ finite and $> L_\infty$.

These encode physical priors on loss surfaces that unconstrained symbolic regression routinely violates. In particular, (A4) must use L_x^∞ rather than L_∞: along a single axis the excess over the joint floor need not vanish, so a power-law rate would not be identifiable against L_∞.

4.3 Boosted construction

Code skeleton: `src/search/baselines/`, `src/search/residuals/`, `src/search/expression/`, `src/search/boosting/`, `src/search/soft_path/`, `src/search/hard_path/`

The ensemble is built stagewise, $\hat{L}^{(j)} = \hat{L}^{(j-1)} + \hat{L}_j$, under an *admissibility invariant*: $\hat{L}^{(j)} \in \mathcal{S}$ for every $j = 0, \ldots, K$ (Section 4.2). Throughout, $\hat{L}_j$ (subscript) is the stage-j correction and $\hat{L}^{(j)}$ (superscript) is the cumulative ensemble through stage j, so in particular $\hat{L}^{(0)} = \hat{L}_0$. Stage 0 is a generalized Chinchilla law, fit by nonlinear least squares with positive parameter constraints and admissible by construction with joint floor $L_\infty = E$ and stage-0 asymptotic exponents $c_N^{(0)} = -\alpha$, $c_D^{(0)} = -\beta$ (the rates c_x in (A4) of Section 4.2 along N and D). Concrete instances include a 2D law in (N, D) and a 4D law that adds two hyperparameter axes (e.g. weight decay and learning rate). The 4D hyperparameter terms may be power laws of the same form as in (3), or—matching U-shaped loss in lr/wd—quadratic penalties in $\log \phi_{\text{lr}}(\text{lr})$ and $\log \phi_{\text{wd}}(\text{wd})$ (shape axioms still act only on N and D):

$$\hat{L}^{(0)} \;=\; \hat{L}_0 \;=\; \frac{A}{N^\alpha} + \frac{B}{D^\beta} + \sum_{i=1}^{m} \frac{C_i}{\phi_i(h_i)^{\gamma_i}} + E, \qquad A, B, C_i, E, \alpha, \beta, \gamma_i > 0. \tag{3}$$

Each later stage $j \geq 1$ fits Huber pseudo-residuals of the current ensemble, with clipping threshold

$$\delta_j \;\coloneqq\; \text{median}_{\ell=1,\ldots,n} \big| L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell) \big| \tag{4}$$

so that outlier runs cannot dominate:

$$\tilde{r}_j^{(\ell)} \;\coloneqq\; \begin{cases} L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell) & \text{if } \big|L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell)\big| \leq \delta_j, \\ \delta_j \cdot \text{sign}\big(L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell)\big) & \text{otherwise.} \end{cases} \tag{5}$$

Corrections are sought by genetic-programming search over expression trees with primary operators $\{+, -, \times, \div\} \cup \{\text{pow}_p : p \in \mathcal{P}\}$ for a finite set $\mathcal{P} \subset \mathbb{R} \setminus \{0\}$ (where $\text{pow}_p(z) = |z|^p$ for $z > 0$) and bounded depth; optional unary helpers such as inv/log/sqrt may be enabled for ablations. Trees are evaluated on *log-features* $\log \phi_h(h)$; raw-scale derivatives needed by (A1)–(A2) are recovered by the chain rule (Section 4.4).

4.4 Stagewise admissibility

Code skeleton: `src/constraints/axioms/stage_conditions.py`, `src/constraints/certificates/`

Checking that the full ensemble $\hat{L}^{(j)}$ obeys (A1)–(A6) of Section 4.2 is a continuum statement over $\mathbb{H}$. Admissibility of the update is nonetheless equivalent to local conditions on the *increment*: given

$\hat{L}^{(j-1)} \in \mathcal{S}$ with joint floor L_∞, $\hat{L}^{(j)} = \hat{L}^{(j-1)} + \hat{L}_j$ is admissible *if and only if* $\hat{L}_j$ satisfies, for each $x \in \{N, D\}$,

$$\begin{aligned}
&\text{(i)} \ \frac{\partial \hat{L}_j}{\partial x} \le -\frac{\partial \hat{L}^{(j-1)}}{\partial x} \quad \text{for all } x \ge x_{\min}, \\
&\text{(ii)} \ \frac{\partial^2 \hat{L}_j}{\partial x^2} \ge -\frac{\partial^2 \hat{L}^{(j-1)}}{\partial x^2} \quad \text{for all } x \ge x_{\min}, \\
&\text{(iii)} \ \hat{L}_j(\mathbf{h}) > -\hat{L}^{(j-1)}(\mathbf{h}) \quad \text{for all } \mathbf{h} \in \mathbb{H}, \\
&\text{(iv)} \ \hat{L}^{(j-1)}(\mathbf{h}) + \hat{L}_j(\mathbf{h}) > L_\infty \quad \text{for all } \mathbf{h} \in \mathbb{H}, \\
&\text{(v)} \ \hat{L}_j(\mathbf{h})\big|_{x=t} = O\big(t^{c_{x,j}}\big) \text{ as } t \to \infty \text{ for some } c_{x,j} < c_x^{(0)}, \\
&\text{(vi)} \ \hat{L}_j \in C^\infty \text{ on } [x_{\min}, x_{\max}] \text{ with } \big|\hat{L}_j(x_{\min}, \cdot)\big| < \infty.
\end{aligned} \tag{6}$$

Conditions (i)–(iv) and (vi) are still quantified over continua ((i)–(ii) on the half-line $x \ge x_{\min}$; (iii)–(iv) on $\mathbb{H}$), and (v) is an asymptotic statement: none of these can be decided by sampling finitely many points of $\mathbb{H}$. They become machine-checkable because every GP proposal g is a concrete finite expression tree built from $\{+, -, \times, \div\} \cup \{\mathrm{pow}_p : p \in \mathcal{P}\}$. Write $F \coloneqq \hat{L}^{(j-1)} + g$. Two complementary certificates decide whether g preserves (6): interval checks establish (i)–(iv) and (vi) on the compact box $\mathcal{I}_x$ covering the observed grid, while the structural check decides (v) exactly (and thereby controls the $x \to \infty$ tail that the box does not cover).

(a) Sound interval-arithmetic checks (conditions (i)–(iv), (vi) on $\mathcal{I}_x$). Over the compact box $\mathcal{I}_x \coloneqq [x_{\min}, x_{\max}]$ (other coordinates ranging over their finite H_h), forward-mode automatic differentiation composed with interval arithmetic [11, 12] evaluates g bottom-up on interval operands and returns guaranteed enclosures $\underline{f} \le f \le \overline{f}$ on $\mathcal{I}_x$ for $f \in \{F, \partial F/\partial x, \partial^2 F/\partial x^2\}$, in time linear in the tree size. When g is represented on log-features, DualInterval evaluation is performed in log-coordinates and first/second derivatives are mapped to the raw axis x by the chain rule before applying (7). A candidate is accepted when, for each $x \in \{N, D\}$,

$$\overline{\partial F/\partial x} \le 0, \quad \underline{\partial^2 F/\partial x^2} \ge 0, \quad \underline{F} > L_\infty, \quad \overline{F} < \infty. \tag{7}$$

Because the enclosures are sound, (7) is a *sufficient* certificate that the corresponding continuum conditions hold *everywhere* on $\mathcal{I}_x$, not merely at sampled points: $\overline{\partial F/\partial x} \le 0$ forces $\partial F/\partial x \le 0$ on $\mathcal{I}_x$ (hence (i) on the box), and likewise for (ii); $\underline{F} > L_\infty > 0$ forces both (iii) and (iv) on the configurations covered by the product of boxes; $\overline{F} < \infty$ forces the finiteness half of (vi) (smoothness of g on $\mathcal{I}_x$ follows from the operator set). The test is conservative: a failure of (7) does *not* prove that g fails (6) (interval over-approximation can reject a truly admissible update), but every accepted g is guaranteed to preserve admissibility on $\mathcal{I}_x$. That one-sided decidability is exactly what a hard filter needs, and evaluating (7) on each GP candidate is the inner-loop hot spot, the direct target of the AMA27 performance work. In the primary soft path (Section 4.6), the same enclosures are converted to continuous violation scores rather than used as a reject gate.

(b) Exact structural checks (condition (v), and prerequisites for it). Define the asymptotic exponent $\mathrm{ord}(g, x)$ recursively on the tree: constants and leaves other than x contribute 0; $\mathrm{pow}_p(x)$ contributes p; products add exponents; quotients subtract; sums take the maximum. Then (v) holds with $c_{x,j} = \mathrm{ord}(g, x)$ precisely when $\mathrm{ord}(g, x) < c_x^{(0)}$ for each $x \in \{N, D\}$. We additionally require that each scale variable $x \in \{N, D\}$ appears as a leaf of g (otherwise the exponent in x is undefined or vacuously 0). Both tests are exact, finite walks over a finite tree, with no floating-point search over $\mathbb{H}$.

Algorithm 1 enforces (6) via certificates (a)–(b) at every stage.

Code skeleton: `src/search/boosting/algorithm.py`

Algorithm 1 Shape-constrained boosted symbolic regression

Require: Dataset $\mathcal{D} = \{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^n$, stages K
Ensure: Admissible ensemble $\hat{L}^{(K)} \in \mathcal{S}$
1: Fit $\hat{L}_0$ via (3) on $\mathcal{D}$; set $\hat{L}^{(0)} \leftarrow \hat{L}_0$
2: **for** $j = 1, \ldots, K$ **do**
3: Compute δ_j via (4)
4: Compute $\tilde{r}_j^{(\ell)}$ for all $\ell = 1, \ldots, n$ via (5)
5: Find $\hat{L}_j \in \arg\min_g \frac{1}{n} \sum_{\ell=1}^n \big(\tilde{r}_j^{(\ell)} - g(\mathbf{h}_\ell)\big)^2$ subject to certificates (a)–(b) for (6) ▷ hard filter extension; soft path uses (10)
6: $\hat{L}^{(j)} \leftarrow \hat{L}^{(j-1)} + \hat{L}_j$
7: **end for**
8: **return** $\hat{L}^{(K)}$

4.5 Guarantee

Code skeleton: `src/constraints/guarantee/invariants.py`

If $\hat{L}_0 \in \mathcal{S}$ and every $\hat{L}_j$ is accepted by certificates (a)–(b) of Section 4.4, then for all j: $\hat{L}^{(j)} \in \mathcal{S}$; writing $L_\infty^{(j)}$ for the joint floor of $\hat{L}^{(j)}$, one has $L_\infty^{(j)} = E$; and the asymptotic exponents are preserved,

$$\lim_{t \to \infty} \frac{\log\big(\hat{L}^{(j)}(\mathbf{h})\big|_{x=t} - L_x^\infty\big)}{\log t} \;=\; c_x^{(0)}, \tag{8}$$

where L_x^∞ is the marginal floor of $\hat{L}^{(j)}$ along x (equal to that of $\hat{L}_0$). Boosting therefore improves fit *without* corrupting the interpretable constants researchers actually quote.

4.6 Soft-constrained variant

Code skeleton: `src/search/soft_path/violations.py`, `src/search/soft_path/fitness.py`, `src/search/soft_path/backends.py`

The primary implemented path replaces a hard admissibility reject filter with per-axiom violation scores on the same candidate ensemble $F := \hat{L}^{(j-1)} + g$, folded into the `gplearn` fitness (Section 3.3). Using the interval enclosures of Section 4.4(a) and the structural quantities of (b), define

$$\begin{aligned}
v_{\text{mono}}(g) &= \max_{x \in \{N,D\}} \max\Big(0,\ \overline{\partial F/\partial x}\Big), && \text{(monotonicity gap)},\\
v_{\text{conv}}(g) &= \max_{x \in \{N,D\}} \max\Big(0,\ -\underline{\partial^2 F/\partial x^2}\Big), && \text{(convexity gap)},\\
v_{\text{irred}}(g) &= \max\big(0,\ L_\infty - \underline{F}\big), && \text{(floor gap)},\\
v_{\text{decay}}(g) &= \max_{x \in \{N,D\}} \max\big(0,\ \text{ord}(g,x) - c_x^{(0)} + \varepsilon_{\text{decay}}\big), && \text{(power-law gap)},\\
v_{\text{leaf}}(g) &= \big|\{x \in \{N,D\} : x \notin \text{leaves}(g)\}\big|, && \text{(missing scale leaves)}.
\end{aligned} \tag{9}$$

Here $\varepsilon_{\text{decay}} > 0$ is a fixed margin so that $v_{\text{decay}}(g) = 0$ forces the strict inequality $\text{ord}(g,x) \le c_x^{(0)} - \varepsilon_{\text{decay}} < c_x^{(0)}$ required by (6)(v); $\text{leaves}(g)$ is the set of variable leaves of the expression tree g. Every score is nonnegative, and $v_{\text{mono}} = v_{\text{conv}} = v_{\text{irred}} = v_{\text{leaf}} = 0$ iff the corresponding hard certificate passes (for v_{decay}, zero is sufficient but slightly stronger than (v) because of the margin). Writing a

for an axiom index ranging over $\{\text{mono}, \text{conv}, \text{irred}, \text{decay}, \text{leaf}\}$ and $\lambda_a > 0$ for its penalty weight, the aggregate violation is $V(g) = \sum_a v_a(g)$. Stage j then minimizes the penalized fitness

$$F_j(g) \;=\; \underbrace{\frac{1}{n}\sum_{\ell=1}^{n}\big(\tilde{r}_j^{(\ell)} - g(\mathbf{h}_\ell)\big)^2}_{\text{data fit}} \;+\; \underbrace{\sum_a \lambda_a\, v_a(g)^2}_{\text{axiom penalty}}, \tag{10}$$

where F_j is a scalar fitness (distinct from the ensemble map F above), the first term is mean squared error of g against the Huber pseudo-residuals (5), and the second term penalizes axiom gaps (squared so the penalty gradient vanishes at zero violation). Candidates that already satisfy (6) therefore incur no penalty pressure. This path carries its own guarantee: $V = 0$ at every stage recovers the hard result exactly; the interval-based scores v_{mono}, v_{conv}, v_{irred} upper-bound the corresponding true pointwise violations (while v_{decay} and v_{leaf} are exact), yielding a *measured* admissibility gap rather than silent failure; and for a sufficiently small decay margin every correction that passes the hard admissibility certificates has zero penalty, so the soft search contains the hard one of Section 4.4 when the latter is implemented. Soft scores act as IA proxies for (A1)–(A4) (with positivity (A5) folded into the irreducible-floor gap and (A6) into finiteness of the DualInterval enclosure / smooth operator set). Comparing soft-penalty search ($\lambda_a > 0$) against an unconstrained baseline ($\lambda_a = 0$), and optionally against a true hard reject filter, is one of the empirical questions the project settles.

5 Additional information

5.1 Collaborators welcome

Code skeleton: `src/modeling/` (modeling); `src/search/`, `src/systems/hpc/` (search)

The project splits into two fairly independent tracks, so a teammate could own either one. The *modeling* track ports and benchmarks transformer pretraining on Neocortex's Cerebras CS-3 systems (Section 3.1) and profiles it against our NVIDIA GPU runs (Section 3.2; useful background: PyTorch, or curiosity about the Cerebras software stack). The *search* track ports the genetic-programming symbolic-regression loop of Sections 4.3 and 4.6 to AMA27 (Section 3.1) and profiles the DualInterval certification inner loop of Section 4.4 on AmpereOne against our AMD EPYC (x86) CPU baselines (useful background: C/C++ or Python performance work, or an interest in Arm/HPC profiling); a self-contained slice of it is benchmarking soft-penalty search against $\lambda_a = 0$ and against a hard reject filter (Section 4.4). No prior scaling-law expertise is needed for either; the main thing is an appetite for making code run fast on unusual hardware.

5.2 Deliverables

Code skeleton: `src/systems/pipeline/run_discovery.py`

(1) An admissible boosted scaling law (Sections 4.2 and 4.5) fit on the collected training grid (Section 3.5); (2) a public release of the shape-constrained symbolic-regression library with the interval-arithmetic-certified admissibility checking of Section 4.4; and (3) a short paper on the method of Section 4.

5.3 Code skeleton map

The repository ships a student implementation skeleton under `src/`, grouped into five packages. Blue links open the corresponding paths on GitHub (`main`). Each technical subsection above also points to those files; the table summarizes the map.

Description	Package	Primary paths
Section 4.1	src/scaling/	src/scaling/setup/configuration.py, src/scaling/setup/constants.py
Section 3.5	src/scaling/	src/scaling/data/scaling_dataset.py, src/scaling/data/corpora.py
Section 4.2	src/constraints/	src/constraints/axioms/admissibility.py
Section 4.4	src/constraints/	src/constraints/axioms/stage_conditions.py, src/constraints/certificates/
Section 4.5	src/constraints/	src/constraints/guarantee/invariants.py
Section 4.3, Alg. 1	src/search/	src/search/baselines/, src/search/residuals/, src/search/expression/, src/search/boosting/, src/search/soft_path/, src/search/hard_path/
Section 4.6	src/search/	src/search/soft_path/
Section 3.4	src/modeling/	src/modeling/models/transformer.py
Section 3.1 (Neocortex)	src/modeling/	src/modeling/training/
Sections 3.1 and 3.2 (AMA27/CPU)	src/systems/	src/systems/hpc/
Section 5.2	src/systems/	src/systems/pipeline/run_discovery.py

6 References

[1] Lukas Biewald. Experiment tracking with weights & biases. https://wandb.ai/, 2020. Software available from wandb.com.

[2] Trevor Stephens. gplearn: Genetic programming in Python. https://gplearn.readthedocs.io/, 2015. Software library.

[3] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models, 2023. URL https://arxiv.org/abs/2302.13971.

[4] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2203.03466. arXiv:2203.03466.

[5] Houyi Li, Wenzhen Zheng, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, et al. Predictable scale: Part i, step law: Optimal hyperparameter scaling law in large language model pretraining, 2025. URL https://arxiv.org/abs/2503.04715.

[6] Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, et al. Language models scale reliably with over-training and on downstream tasks, 2024. URL https://arxiv.org/abs/2403.08540.

[7] Wikimedia Foundation. Wikimedia downloads. https://dumps.wikimedia.org, 2023. Wikipedia dumps; also available as https://huggingface.co/datasets/wikimedia/wikipedia.

[8] Together Computer. RedPajama: An open dataset for training large language models. https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T, 2023. Hugging Face dataset.

[9] Guilherme Penedo et al. FineWeb-Edu: Educational subset of FineWeb. https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu, 2024. Hugging Face dataset.

[10] Leibniz Lab. colinear_scaling_models: Scaling-grid checkpoints and loss curves. https://huggingface.co/datasets/leibnitz-lab/colinear_scaling_models, 2025. Hugging Face dataset.

[11] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. Society for Industrial and Applied Mathematics (SIAM), 2009. doi: 10.1137/1.9780898717716. URL https://doi.org/10.1137/1.9780898717716.

[12] G. Kronberger, F. O. de França, B. Burlacu, C. Haider, and M. Kommenda. Shape-constrained symbolic regression: Improving extrapolation with prior knowledge. *Evolutionary Computation*, 30(1):75–98, 2022. doi: 10.1162/evco_a_00294. URL https://doi.org/10.1162/evco_a_00294.

7 Notation

Symbol	Meaning
N, D	Parameter count and training-token count (scale variables).
M	Tokens-per-parameter ratio $M = D/N$ (Section 3.4).
$\mathcal{H} = \{h_1, \ldots, h_m\}$	Remaining hyperparameters; $m = \lvert\mathcal{H}\rvert$.
i	Hyperparameter index, $i = 1, \ldots, m$ (distinct from data-point index ℓ).
H_h	Finite set of admissible values for coordinate $h \in \{N, D\} \cup \mathcal{H}$.
$\phi_h,\ \phi_i$	Strictly monotone preprocessing $\phi_h \colon H_h \to \mathbb{R}_{>0}$; $\phi_i \coloneqq \phi_{h_i}$.
$\mathbb{H}$	Configuration space $\prod_h H_h$.
$\mathbf{h}$	Typical element of $\mathbb{H}$ (coordinates N, D, h_i).
$L,\ \hat{L}$	True validation loss and its approximation.
$\mathcal{D},\ n,\ \ell$	Labeled dataset $\{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^{n}$; $n = \lvert\mathcal{D}\rvert$; data-point index ℓ.
x	Generic scale variable, $x \in \{N, D\}$.
t	Dummy limit variable along a scale axis (e.g. $\hat{L}(\mathbf{h})\rvert_{x=t}$).
$x_{\min},\ x_{\max}$	$\min H_x$ and $\max H_x$.
$\mathcal{S}$	Set of admissible scaling laws (Section 4.2).
$L_\infty,\ L_\infty^{(j)}$	Joint irreducible-loss floor; joint floor of $\hat{L}^{(j)}$ ($L_\infty^{(j)} = E$ under the hard path).
L_x^∞	Marginal floor along axis x (1).
$c_x,\ c_x^{(0)}$	Asymptotic power-law exponent along x (A4); stage-0 value $c_N^{(0)} = -\alpha$, $c_D^{(0)} = -\beta$.
$j,\ K$	Boosting stage index; number of correction stages after stage 0 (Algorithm 1).
$\hat{L}_j,\ \hat{L}^{(j)}$	Stage-j correction (subscript) and cumulative ensemble through stage j (superscript); $\hat{L}^{(0)} = \hat{L}_0$.
$A, B, C_i, E, \alpha, \beta, \gamma_i$	Positive coefficients/exponents of the stage-0 Chinchilla law (3); joint floor $L_\infty = E$.
δ_j	Huber clipping threshold at stage j (4).
$\tilde{r}_j^{(\ell)}$	Huber pseudo-residual for data point ℓ at stage j (5).
$\mathcal{P},\ p$	Finite set of admissible powers $\mathcal{P} \subset \mathbb{R} \setminus \{0\}$; generic element p.
pow_p	Power primitive $\mathrm{pow}_p(z) = \lvert z\rvert^p$ for $z > 0$.
$g,\ F$	Candidate expression tree; ensemble-plus-candidate $F = \hat{L}^{(j-1)} + g$.
$c_{x,j},\ \mathrm{ord}(g, x)$	Asymptotic exponent of correction/candidate g along x; $c_{x,j} = \mathrm{ord}(g, x)$ in (6)(v).
$\mathcal{I}_x$	Compact certification box $[x_{\min}, x_{\max}]$ (Section 4.4).
$\underline{f},\ \overline{f}$	Interval-arithmetic lower/upper enclosures of f on $\mathcal{I}_x$ (global over that box for fixed x; soft scores take $\max_{x \in \{N,D\}}$ of the per-axis gaps).
$a,\ v_a(g),\ V(g)$	Soft-path axiom index $a \in \{\mathrm{mono}, \mathrm{conv}, \mathrm{irred}, \mathrm{decay}, \mathrm{leaf}\}$; per-axiom violation score; aggregate $V = \sum_a v_a$ (9).
$\varepsilon_{\mathrm{decay}}$	Positive margin in v_{decay} so zero score implies strict power-law decay (9).
$\mathrm{leaves}(g)$	Set of variable leaves of expression tree g.
λ_a	Positive penalty weight for axiom a.
$F_j(g)$	Soft-path penalized fitness at stage j (10) (distinct from F).